Final Project

Monday, October 20, 2025 11:25 AM

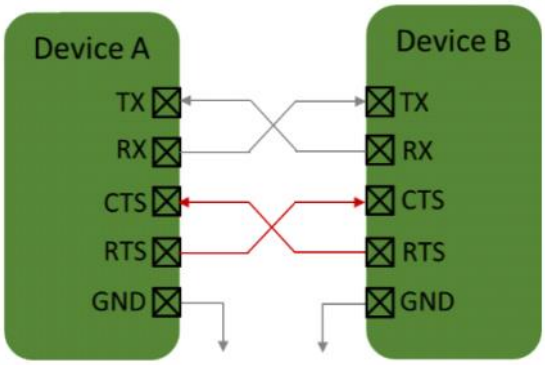
Lab 10: Full-duplex RGB image data exchange between FPGA and PC over UART with a back-pressure controlled data-pipe

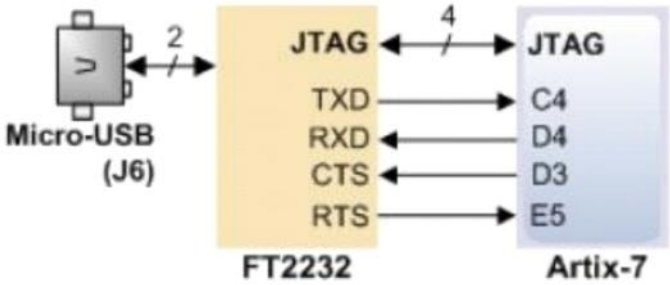
Objective

The objective of this project is to design and implement a complete system that is able to receive and transmit an RGB image of selectable dimensions over UART to and from a PC.
Data paths should be designed for maximum throughput to reduce transmit times as much as possible.
The overall data-flow process will be based on handshake protocols to avoid overflow and loss of packet data. Back-pressure should be timed efficiently to maximize the bus utilization ratios.
The code must be in SystemVerilog, parametrized and generate based wisely.
Tis final project is the ending effort of your previous labs accumulated.
All you have to do is some final adjustments and to check the End-to-End data flow.

Requirements

1. **PLL**
   - The UART logic will be driven with high clock frequencies to raise the baud
   - Use a PLL/MMCM if clock frequencies other than the system clock of 100M is required
2. **Reset Synchronizers**
   - Manage the reset tree to avoid any RDC issues
3. **Clock multiplexer**
   - If multiple clocks are used, are clock muxing is applied?
   - If that is positive, then is it a glitch-free mux?
   - Make sure clock muxing is not problematic
4. **UART TX**
   - UART TX module should be broken to a MAC and a PHY submodules
   - Document the message/signal exchange between the MAC and PHY
   - Parity bit should be added to provide data robustness against errors
   - Upon a successful message transmit the counters indicating the row/col indices should be updated
   - Stress the UART baud rate to be above 5MHz
   - Use the RTS/CTS for proper backpressure
5. **UART RX**
   - UART RX module should also be broken to a MAC and a PHY submodules
   - The RX PHY has to check the parity correctness
     - On error increment the number of faulty frames in RGF
     - Do not forward a faulty frame onwards to the MAC
     - Signal the MAC that a phy error occurred and soft-reset the MAC
   - Use the RTS/CTS for proper backpressure
   - RX baud rate must be equal to the TX baud rate!
6. **Async FIFO**
   - If you managed to have two different clocks running the system
     - High-frequency PLL generated clock for UART logic and front-end part
     - System-clock for the SRAM and DMA blocks
   - Then 3 async FIFO(s) are required to handle the data crossing between the two clock domains. One per color channel.
   - Will hold the incoming values from the RGB SRAMs
   - 32 bits width to hold 4 pixel values per row
     - R FIFO: R0, R1, R2, R3
     - G FIFO: G0, G1, G2, G3
     - B FIFO: B0, B1, B2, B3
7. **Sequencer - TX Part**
   - The Sequencer will read in a burst of 4 beats from the 3 SRAMs in a round-robin style
     - R FIFO will initiate a burst of 4 to read from the R SRAM:
       - R0, R1, R2, R3
       - R4, R5, R6, R7
       - R8, R9, R10, R11
       - R12, R13, R14, R15
     - Next will do the G, and B FIFOs respectively, and cycle back to the R again
     - Bursts will be over AHB
   - If the sequencer cannot handle the data rate, fall to a read from SRAMs every 4th cycle
8. **Message Composer**
   - Messages from FPGA to PC must be sent over UART following our agreed protocol
   - Once message is created from the values, it must be sampled and provided to the UART TX MAC and cannot fetch any new data until the UART MAC signals back the message was correctly sent
9. **Sequencer - RX Part**
   - The sequencer should split the received message to different push sequences
     - Start with the R FIFO push, next the G and lastly the B
   - FIFO push (R0, G0, B0), (R1, G1, B1), …
10. **Parser + Classifier**
    - The incoming messages should go through message parser and message classifier
    - The Parser should be pure combo that parse the message fast to determine if it is a legal message or not
    - The classifier classifies the message type and extracted values
    - The classifier will sample the values and hold them till the sequencer pushed the data to the requested addresses
11. **Messages**
    - Our system supports a number of messages
      - Register Write [PC -> FPGA]
        - {W<A2, A1, A0>, V <Ø, DH1, DH0>, V <Ø, DL1, DL0>}
      - Register Read [PC -> FPGA]
        - {R<A2, A1, A0>}
      - Single Pixel Write [PC -> FPGA]
        - {W<A2, A1, A0>, P<R, G, B>}
      - Single Pixel Read [FPGA -> PC]
        - {R<R2,R1, R0>, C<C2, C1, C0>, P<R, G, B>}
      - Image Burst Write [PC -> FPGA]
        - {I<Ø, Ø, Ø>, H<H2, H1, H0>, W<W2,W1,W0>}
          - Will enable opcode bypass for the duration of HxW pixels
        - {<R0, G0, B0, R1>, <G1, B1, R2, G2>, <B2, R3, G3, B3>}
          - Overall 3 groups of 32bits -> 4 pixels in one message
      - Image Burst Read [PC -> FPGA]
        - {R<A2, A1, A0>, H<H2, H1, H0>, W<W2,W1,W0>}
        - FPGA will reply with repetitive messages of 4 pixels for HxW pixels:





        - {<R0, G0, B0, R1>, <G1, B1, R2, G2>, <B2, R3, G3, B3>}

12. **Testbench**:
    - Use timescale 1ns / 1ps.
    - Use defines or parameters for controlling real HW or Simulation
13. **Physical Implementation:**
    - Set the XDC file with the abovementioned I/Os
    - Connect the dut ports appropriately
    - Synthesize, PnR, Gen bitstream
    - Test on the Nexys board
14. **Deliverables**:
    - Vivado Project
    - Verilog code for the design and the testbenches.
    - Validated Bitstream
    - Simulation waveform screenshots (paste into your report)
    - A report with:
        - Blocks diagram
        - Sketch and describe the <u>data pipe flow</u> entirely! - Most important
        - Address space mapping - Keep some margin
        - Register file documentation with informative description
        - FSM design diagrams
        - Lint schematics and clean report
        - Synthesis & Implementation reports are clean
        - Resource expectations & Resources usage report
            - are the reported resources meet expectations?
        - Design process and simulation results